Oexa Dental Clinic System – Database Design

1 Main Database Components

The database schema includes the following components:

Tables (entities)
Attributes (columns)
Primary Keys
Foreign Keys
Relationships
Constraints
Junction tables for many-to-many relationships

The main entities of the system are:

Users
Patients
Dentists
Services
Appointments
Treatments
Prescriptions
Medications
Prescription_Medication
Receipts
Payments
Promotions
Service_Promotions
CustomerCategories
Promotion_Categories
Ratings

2 Entity Relationship Overview

The main relationships in the system are as follows:

A User may be a Patient
A User may be a Dentist
A Patient can have many Appointments
A Dentist can have many Appointments
A Service can be associated with many Appointments
An Appointment may have one Treatment
A Treatment may have many Prescriptions
A Prescription can include many Medications
An Appointment may generate one Receipt
A Receipt can have many Payments
A Promotion can be applied to many Services
A Promotion can be applied to many Customer Categories
A Customer Category can contain many Patients
An Appointment may have one Rating

3 Entities and Attributes

3.1 Users

The Users table stores the common account information of all system users.
Attributes:

- user_id (Primary Key)
- full_name
- email
- password_hash
- role
- created_at

3.2 Patients

The Patients table stores patient-specific information.
Attributes:

- patient_id (Primary Key)
- user_id (Foreign Key)
- phone
- date_of_birth
- address
- category_id (Foreign Key)

3.3 Dentists

The Dentists table stores dentist-specific information.
Attributes:

- dentist_id (Primary Key)
- user_id (Foreign Key)
- specialization
- license_number

3.4 Services

The Services table stores all dental services offered by the clinic.
Attributes:

- service_id (Primary Key)
- name
- description
- base_price
- duration_minutes

3.5 Appointments

The Appointments table stores booking details between patients and dentists.
Attributes:

- appointment_id (Primary Key)
- patient_id (Foreign Key)
- dentist_id (Foreign Key)
- service_id (Foreign Key)
- appointment_datetime
- status
- notes
- created_at

3.6 Treatments

The Treatments table stores information about treatment performed during an appointment.
Attributes:

- treatment_id (Primary Key)
- appointment_id (Foreign Key)
- description
- treatment_date
- status

3.7 Prescriptions

The Prescriptions table stores prescribed medication information related to treatment.
Attributes:

- prescription_id (Primary Key)
- treatment_id (Foreign Key)
- notes
- created_at

3.8 Medications

The Medications table stores medication details.
Attributes:

- medication_id (Primary Key)
- name
- description

3.9 Prescription_Medication

This is a junction table that connects prescriptions and medications.
Attributes:

- id (Primary Key)
- prescription_id (Foreign Key)
- medication_id (Foreign Key)
- dosage
- instructions

3.10 Receipts

The Receipts table stores financial receipt information after appointment completion.
Attributes:

- receipt_id (Primary Key)
- appointment_id (Foreign Key)
- total_amount
- created_at
- status

3.11 Payments

The Payments table stores payment records related to receipts.
Attributes:

- payment_id (Primary Key)
- receipt_id (Foreign Key)
- amount
- payment_method
- status
- payment_date

3.12 Promotions

The Promotions table stores promotional campaigns created by the marketer.
Attributes:

- promotion_id (Primary Key)
- name
- discount_percentage
- start_date
- end_date
- status

3.13 Service_Promotions

This is a junction table between services and promotions.
Attributes:

- id (Primary Key)
- service_id (Foreign Key)
- promotion_id (Foreign Key)

3.14 CustomerCategories

The CustomerCategories table stores patient categories such as VIP, Regular, and New.
Attributes:

- category_id (Primary Key)
- name
- description

3.15 Promotion_Categories

This is a junction table between promotions and customer categories.
Attributes:

- id (Primary Key)
- promotion_id (Foreign Key)
- category_id (Foreign Key)

3.16 Ratings

The Ratings table stores feedback given by patients after appointments.
Attributes:

- rating_id (Primary Key)
- appointment_id (Foreign Key)
- patient_id (Foreign Key)
- dentist_id (Foreign Key)
- rating_value
- comment
- created_at

4 Relational Schema

4.1 Users

Users(
user_id PK,
full_name,
email,
password_hash,
role,
created_at
)

4.2 Patients

Patients(
patient_id PK,
user_id FK → Users(user_id),
phone,
date_of_birth,
address,
category_id FK → CustomerCategories(category_id)
)

4.3 Dentists

Dentists(
dentist_id PK,
user_id FK → Users(user_id),
specialization,
license_number
)

4.4 Services

Services(
service_id PK,
name,
description,
base_price,
duration_minutes
)

4.5 Appointments

Appointments(
appointment_id PK,
patient_id FK → Patients(patient_id),
dentist_id FK → Dentists(dentist_id),

service_id FK → Services(service_id),
appointment_datetime,
status,
notes,
created_at
)

4.6 Treatments

Treatments(
treatment_id PK,
appointment_id FK → Appointments(appointment_id),
description,
treatment_date,
status
)

4.7 Prescriptions

Prescriptions(
prescription_id PK,
treatment_id FK → Treatments(treatment_id),
notes,
created_at
)

4.8 Medications

Medications(
medication_id PK,
name,
description
)

4.9 Prescription_Medication

Prescription_Medication(
id PK,
prescription_id FK → Prescriptions(prescription_id),
medication_id FK → Medications(medication_id),
dosage,
instructions
)

4.10 Receipts

Receipts(
receipt_id PK,
appointment_id FK → Appointments(appointment_id),
total_amount,
created_at,

status
)

4.11 Payments

Payments(
payment_id PK,
receipt_id FK → Receipts(receipt_id),
amount,
payment_method,
status,
payment_date
)

4.12 Promotions

Promotions(
promotion_id PK,
name,
discount_percentage,
start_date,
end_date,
status
)

4.13 Service_Promotions

Service_Promotions(
id PK,
service_id FK → Services(service_id),
promotion_id FK → Promotions(promotion_id)
)

4.14 Customer_Categories

CustomerCategories(
category_id PK,
name,
description
)

4.15 Promotion_Categories

Promotion_Categories(
id PK,
promotion_id FK → Promotions(promotion_id),
category_id FK → CustomerCategories(category_id)
)

4.16 Ratings

Ratings(
rating_id PK,
appointment_id FK → Appointments(appointment_id),

patient_id FK → Patients(patient_id),
dentist_id FK → Dentists(dentist_id),
rating_value,
comment,
created_at
)

5. Relationship Cardinalities with Mandatory and Optional Participation

5.1 Users and Patients

User (1..1) —— (0..1) Patient

Meaning:
Every Patient must be linked to exactly one User
A User may or may not be a Patient

5.2 Users and Dentists

User (1..1) —— (0..1) Dentist

Meaning:
Every Dentist must be linked to exactly one User
A User may or may not be a Dentist

5.3 Patients and Appointments

Patient (1..1) —— (0..M) Appointments

Meaning:
Every Appointment must belong to one Patient
A Patient may have zero, one, or many Appointments

5.4 Dentists and Appointments

Dentist (1..1) —— (0..M) Appointments

Meaning:
Every Appointment must belong to one Dentist
A Dentist may have zero, one, or many Appointments

5.5 Services and Appointments

Service (1..1) —— (0..M) Appointments

Meaning:
Every Appointment must be connected to one Service
A Service may appear in zero, one, or many Appointments

5.6 Appointments and Treatments

Appointment (1..1) —— (0..1) Treatment

Meaning:
Every Treatment must belong to one Appointment
An Appointment may or may not have a Treatment yet

5.7 Treatments and Prescriptions

Treatment (1..1) —— (0..M) Prescriptions

Meaning:
Every Prescription must belong to one Treatment
A Treatment may have zero, one, or many Prescriptions

5.8 Prescriptions and Medications

Prescription (1..M) —— (1..M) Medication

Meaning:
A Prescription must contain at least one Medication
A Medication can appear in one or many Prescriptions

This many-to-many relationship is resolved using the Prescription_Medication table.

5.9 Appointments and Receipts

Appointment (1..1) —— (0..1) Receipt

Meaning:
Every Receipt must belong to one Appointment
An Appointment may or may not have a Receipt

5.10 Receipts and Payments

Receipt (1..1) —— (0..M) Payments

Meaning:
Every Payment must belong to one Receipt
A Receipt may have zero, one, or many Payments

5.11 Promotions and Services

Promotion (0..M) —— (0..M) Service

Meaning:
A Promotion may apply to zero, one, or many Services
A Service may have zero, one, or many Promotions

This many-to-many relationship is resolved using the Service_Promotions table.

5.12 CustomerCategories and Patients

CustomerCategory (1..1) —— (0..M) Patients
Patient (0..1) —— (1..1) CustomerCategory

Meaning:
A Patient may or may not belong to a Customer Category
A Customer Category may contain zero, one, or many Patients

5.13 Promotions and CustomerCategories

Promotion (0..M) —— (0..M) CustomerCategory

Meaning:
A Promotion may apply to zero, one, or many Customer Categories
A Customer Category may have zero, one, or many Promotions

This many-to-many relationship is resolved using the Promotion_Categories table.

5.14 Appointments and Ratings

Appointment (1..1) —— (0..1) Rating

Meaning:
Every Rating must belong to one Appointment
An Appointment may or may not receive a Rating

6. SQL Schema

```sql
CREATE TABLE Users (
    user_id INT PRIMARY KEY AUTO_INCREMENT,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    role VARCHAR(20) NOT NULL,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

```sql
CREATE TABLE CustomerCategories (
    category_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(50) NOT NULL UNIQUE,
    description VARCHAR(255)
);
```

```sql
CREATE TABLE Patients (
    patient_id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT NOT NULL UNIQUE,
    phone VARCHAR(20),
    date_of_birth DATE,
    address VARCHAR(255),
    category_id INT,
    FOREIGN KEY (user_id) REFERENCES Users(user_id),
    FOREIGN KEY (category_id) REFERENCES CustomerCategories(category_id)
);
```

```sql
CREATE TABLE Dentists (
    dentist_id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT NOT NULL UNIQUE,
    specialization VARCHAR(100) NOT NULL,
    license_number VARCHAR(100) NOT NULL UNIQUE,
    FOREIGN KEY (user_id) REFERENCES Users(user_id)
);
```

```sql
CREATE TABLE Services (
    service_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    description TEXT,
    base_price DECIMAL(10,2) NOT NULL,
    duration_minutes INT NOT NULL
);
```

```sql
CREATE TABLE Appointments (
    appointment_id INT PRIMARY KEY AUTO_INCREMENT,
    patient_id INT NOT NULL,
    dentist_id INT NOT NULL,
    service_id INT NOT NULL,
    appointment_datetime DATETIME NOT NULL,
    status VARCHAR(30) NOT NULL DEFAULT 'Requested',
    notes TEXT,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (patient_id) REFERENCES Patients(patient_id),
    FOREIGN KEY (dentist_id) REFERENCES Dentists(dentist_id),
    FOREIGN KEY (service_id) REFERENCES Services(service_id)
);
```

```sql
CREATE TABLE Treatments (
    treatment_id INT PRIMARY KEY AUTO_INCREMENT,
    appointment_id INT NOT NULL UNIQUE,
    description TEXT NOT NULL,
    treatment_date DATE NOT NULL,
    status VARCHAR(30) NOT NULL,
    FOREIGN KEY (appointment_id) REFERENCES Appointments(appointment_id)
);
```

```sql
CREATE TABLE Prescriptions (
    prescription_id INT PRIMARY KEY AUTO_INCREMENT,
    treatment_id INT NOT NULL,
    notes TEXT,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (treatment_id) REFERENCES Treatments(treatment_id)
);
```

```sql
CREATE TABLE Medications (
    medication_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    description TEXT
);
```

```sql
CREATE TABLE Prescription_Medication (
    id INT PRIMARY KEY AUTO_INCREMENT,
    prescription_id INT NOT NULL,
    medication_id INT NOT NULL,
    dosage VARCHAR(100) NOT NULL,
    instructions VARCHAR(255),
    FOREIGN KEY (prescription_id) REFERENCES Prescriptions(prescription_id),
    FOREIGN KEY (medication_id) REFERENCES Medications(medication_id)
);
```

```sql
CREATE TABLE Receipts (
    receipt_id INT PRIMARY KEY AUTO_INCREMENT,
    appointment_id INT NOT NULL UNIQUE,
    total_amount DECIMAL(10,2) NOT NULL,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    status VARCHAR(30) NOT NULL DEFAULT 'Draft',
    FOREIGN KEY (appointment_id) REFERENCES Appointments(appointment_id)
);
```

```sql
CREATE TABLE Payments (
    payment_id INT PRIMARY KEY AUTO_INCREMENT,
    receipt_id INT NOT NULL,
    amount DECIMAL(10,2) NOT NULL,
    payment_method VARCHAR(50) NOT NULL,
    status VARCHAR(30) NOT NULL DEFAULT 'Pending',
    payment_date DATETIME,
    FOREIGN KEY (receipt_id) REFERENCES Receipts(receipt_id)
);
```

```sql
CREATE TABLE Promotions (
    promotion_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    discount_percentage DECIMAL(5,2) NOT NULL,
    start_date DATE NOT NULL,
    end_date DATE NOT NULL,
    status VARCHAR(30) NOT NULL
);
```

```sql
CREATE TABLE Service_Promotions (
    id INT PRIMARY KEY AUTO_INCREMENT,
    service_id INT NOT NULL,
    promotion_id INT NOT NULL,
    FOREIGN KEY (service_id) REFERENCES Services(service_id),
    FOREIGN KEY (promotion_id) REFERENCES Promotions(promotion_id)
);
```

```sql
CREATE TABLE Promotion_Categories (
    id INT PRIMARY KEY AUTO_INCREMENT,
    promotion_id INT NOT NULL,
    category_id INT NOT NULL,
    FOREIGN KEY (promotion_id) REFERENCES Promotions(promotion_id),
    FOREIGN KEY (category_id) REFERENCES CustomerCategories(category_id)
);
```

```sql
CREATE TABLE Ratings (
    rating_id INT PRIMARY KEY AUTO_INCREMENT,
    appointment_id INT NOT NULL UNIQUE,
    patient_id INT NOT NULL,
    dentist_id INT NOT NULL,
    rating_value INT NOT NULL,
    comment TEXT,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (appointment_id) REFERENCES Appointments(appointment_id),
    FOREIGN KEY (patient_id) REFERENCES Patients(patient_id),
    FOREIGN KEY (dentist_id) REFERENCES Dentists(dentist_id)
);
```

7 Normalization

The database design is normalized up to Third Normal Form (3NF).

First Normal Form (1NF)
All tables contain atomic values, and there are no repeating groups.

Second Normal Form (2NF)
All non-key attributes are fully dependent on the primary key.

Third Normal Form (3NF)
There are no transitive dependencies, and each non-key attribute depends only on the key.

This normalization improves data consistency, reduces redundancy, and makes the database easier to maintain.